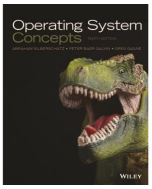


Operating Systems 2025

U3680

National Taipei University

Midterm Exam [Chapter 1 - Chapter 5]

---

1. Answer **True** or **False** for the following statements. **If you choose False, you should give an explanation.** Be sure to include the **question number** with each answer (e.g., (1) ___).

   (1) Cache can always improve access time when retrieving data from the main memory. (3%)
   False. Cache cannot improve access time and will introduce overhead if the program has no locality.

   (2) Direct memory access (DMA) enhances CPU throughput by speeding up each interrupt service routine. (3%)
   False. DMA enhances CPU throughput by reducing the frequency of interrupt.

   (3) System calls are usually implemented by high-level languages to improve portability. (3%)
   False. System calls are usually implemented by assembly for better performance.

   (4) Multiple API functions can use the same system call. (3%)
   True.

   (5) A micro-kernel OS usually has good performance due to its small kernel size. (3%)
   False. In a micro-kernel OS, subsystems operate as user-space programs and communicate via message passing, which typically results in lower performance.

   (6) Message passing is slow because each communication requires OS intervention. (3%)
   True.

   (7) Multiple threads of a process can share the register sets. (3%)
   False. Each thread can run different piece of code. Thus, the register sets cannot be shared.

   (8) Because I/O operations are time-consuming, I/O bound processes are usually placed in lower-priority queues in a multi-level feedback queue scheduling algorithm. (3%)
   False. I/O-bound processes have shorter CPU bursts and are usually assigned to higher-priority queues based on the principle of SJF.

   (9) Copy-on-write helps minimize initial memory usage during process forking. (3%)
   True.

   (10) When a round-robin algorithm uses a time quantum longer than all processes' CPU bursts, it becomes a first-come-first-serve algorithm. (3%)
   True.

2. Should a high-speed device, such as a network interface card, use interrupt-based I/O? Explain your reasoning. (6%)
   No. If the device operates at high speed, it will generate frequent interrupts, keeping the CPU busy with interrupt handling. A better approach is to periodically check the device status instead (polling).

3. What are the limitations of batch systems? How have later OS designs overcome them? (6%)

   - Batch systems handle one job at a time, which often leaves the CPU idle. This limitation can be mitigated by loading multiple processes into memory (multi-programming).
   - Batch systems do not support user interaction. This limitation can be overcome by dividing CPU time into small slices and frequently switching between jobs (time-sharing).

4. Analyze how the frequency of context switching affects performance when the interval is either too short or too long. (4%)
   Because context switching is pure overhead, too much context switching degrades performance due to overhead, while too little causes slow user response.

5. Consider the following processes entering a system:

Process	Arrival	Burst
P_1	0	9
P_2	2	6
P_3	2	5
P_4	5	1

   (a) If a non-preemptive Shortest-Job-First (SJF) scheduling algorithm is used, show the execution order of the processes using a Gantt chart. (3%)

   (b) Show the Gantt chart if a preemptive SJF algorithm is used instead. (3%)

   (c) Considering questions (a) and (b), which scheduling algorithm is more efficient in terms of average waiting time? (4%)
   The average waiting time for non-preemptive and preemptive SJF are 6.25 and 4.75, respectively. Preemptive SJF proves to be more efficient due to its increased flexibility in scheduling decisions.

6. Briefly describe the purposes of long-term, short-term, and medium-term schedulers. (6%)

   - Long-term scheduler selects which processes should be loaded into memory and brought into the ready queue.
   - Short-term scheduler selects which process should be executed and allocated CPU resources.
   - Medium-term scheduler selects which processes should be swapped in/out of memory.

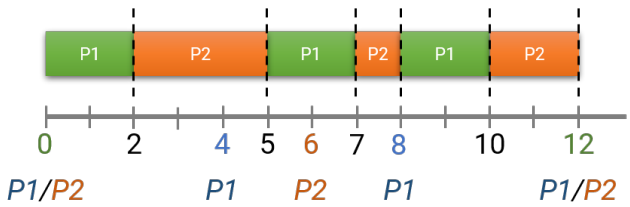
7. Provide two advantages of thread pools. (6%)
   - Handling a request with an existing thread is typically quicker than creating a new one.
   - It ensures that the number of threads in the application is restricted to the pool size.

8. Answer the following questions using Amdahl's Law. You should include the calculation process.
   (a) What is the speedup achieved when running a program with 40% parallelism on 2 cores? (4%)
   1.25
   (b) Can a program with 60% parallelism achieve a speedup of at least 2 times when executed on 4 cores? Explain your reasoning. (4%)
   No. Based on Amdahl's Law, the highest achievable speedup is 1.82.

9. Load balancing can be achieved through push or pull migration. Provide the reasons why a system often implements both strategies. (6%)
   This is because a system selects either push or pull migration based on CPU availability. When most CPUs are busy, pull migration is more effective. Conversely, when most CPUs are idle, push migration is the better option.

10. The following real-time processes are being scheduled using the Early Deadline First (EDF) algorithm. Please show the scheduling order of the first 12 time units using a Gantt chart. If two processes have the same deadline, execute P_1 first. **Please label the time when process changes**. (6%)

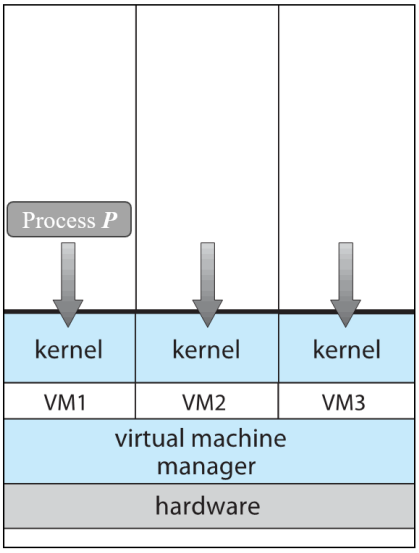
Process	Ready	Execution	Period
P_1	0	2	4
P_2	0	3	6



11. Based on the virtual machine illustrated in Fig.1(a), describe what happens when process P attempts to execute a privileged instruction. (6%)
   Process P initiates a system call through a software interrupt, which is handled by the kernel of the virtual machine it's running on (VM1). When the VM's kernel attempts to execute a privileged instruction, the operation fails because it runs in user space. As a result, the host OS receives another software interrupt triggered by this failure. Recognizing that the issue originates from the virtual machine, the host OS executes the privileged instruction and returns the result.

12. What are the outputs of the program shown in Fig.1(b), assuming the 'helloworld' system program prints the line "hello world!" to the screen? (6%)
   1 2 3 4
   hello world!
   2 4 6 8
   program finished

(a)



(b)

```
#include <sys/types.h>
…

#define SIZE 4
int data[SIZE] = {2, 4, 6, 8}

int main() {
    pid_t pid;
    pid = fork();
    if (pid > 0) {
        wait(NULL);
        for (int i = 0 ; i < SIZE ; ++i)
            printf("%d ", data[i]);
        printf("\n");
    }
    else if (pid == 0) {
        for (int i = 0 ; i < SIZE ; ++i) {
            data[i] /= 2;
            printf("%d ", data[i]);
        }
        printf("\n");
        execlp("helloworld", "helloworld", NULL, NULL);
        printf("child complete");
    }
    printf("program finished\n");
    return 0;
}
```

Figure 1: (a) Virtual machine for Question 11. (b) Program for Question 12.